Project Info

Team Name: Aunt Leah B
Team Members:

- Shreyan Das (51692796) - Project Manager
- Michael Sam (23472046) - QA Lead
- Kyaw Min Oo (51290849) - UI/UX Lead
- David Falade (78099298) - DevOps Lead
- Aryan Saini (77584373) - Client Liaison

Client name: David Savory/ Aunt Leah's
Github Repo: https://github.com/aunt-leahs-b-cms/aunt-leah-b
Project Management Software: https://app.clickup.com/9017781312/v/b/li/901711006277
Document: Version 1

Testing Strategy

Unit Testing
We test individual tRPC backend procedures in isolation by mocking the Prisma database client. This ensures no real Postgres database calls are made during unit tests. Our tests cover CRUD operations for each route, user permission logic, and form validation to verify that business rules are enforced at the API level.
Execution: Vitest with mocked Prisma client (automated)

Integration Testing
We test connected components and services with a live test database to verify data flows correctly between layers, in other words, it focuses on critical paths rather than exhaustive edge cases like our unit tests (which mock the database). Our integration tests will execute actual CRUD operations to ensure our tRPC procedures, database queries, and business logic work together as expected.
Execution: Automatic through local mock database with vitestt, owned by all developers

System/E2E Testing
We validate the fully integrated application (database, network, API calls and user interfaces) by simulating real user scenarios from start to finish. The QA lead assumes the roles of different user types (e.g., admin, case worker, manager) and executes critical workflows through the actual UI to verify that the system behaves as expected in a production-like environment.
Execution: Manual testing that is executed by QA (Michael)

Smoke Testing
After each new deployment, we run a rapid set of minimal checks to verify that the application's

most critical paths are functional (e.g., users can log in, load the dashboard, perform a basic CRUD operation). This ensures the build is stable enough for more thorough testing.
Execution: Automated via Github actions

Accessibility Testing

We verify that our application can be effective to people mainly visual impairments such as color blindness. This includes sufficient color contrast and visible focus indicators.
Execution: Manual testing using browser tools (e.g. Chrome Lighthouse) and keyboard-only navigation, executed by PM (Shreyan) and QA (Michael)

Prioritization

We are prioritizing End-to-End testing next. While unit tests are already implemented and provide confidence in our backend logic, our MVP testing evaluation revealed edge cases in the UI that need to be addressed. Manual E2E testing will allow us to identify and fix these user-facing issues before investing time in integration or automated smoke tests.

Testing Overview

Workflow	Test Layer	Evidence Location	Gap
View/Edit/Search Client	**Unit Test**	**https://github.com/aunt-leahs-b-cms/aunt-leah-b/blob/main/server/routers/__tests__/client.test.ts**	**- No tests for assertCanAccessClientById helper or scenarios where users lack program access (FORBIDDEN cases) - No tests confirming that invalid inputs (bad email, invalid enums) properly throw validation errors - Tests confirm audit log is called but not that it captures the correct change data**
Authentication / MFA / Admin User Management / Program Security	**Unit Tests**	**https://github.com/aunt-leahs-b-cms/aunt-leah-b/blob/main/server/routers/__tests__/auth.test.ts https://github.com/aunt-leahs-b-cms/aunt-leah-b/blob/main/server/routers/__tests__/adminAuth.test.ts https://github.com/aunt-leahs-b-cms/aunt-leah-b/blob/main/server/helper/__tests__/programAccess.test.ts**	**- No automated E2E coverage for full auth/admin UI workflows - No browser-based tests for onboarding, MFA setup, or admin dashboard interactions**
Goal Setting for a Client	**Unit Tests**	**https://github.com/aunt-leahs-b-cms/aunt-leah-b/blob/main/server/routers/__tests__/goal.test.ts**	
Intake a client through the form	**Unit Tests**	**https://github.com/aunt-leahs-b-cms/aunt-leah-b/blob/main/server/routers/__tests__/intake.test.ts**	**- Form doesnt validate dates (e.g DOB in the future, DOE in the past) - Race Condition may**

			occur while creating client for the first time
View/Edit/Create Case Notes	**Unit Tests**	**https://github.com/aunt-leahs-b-cms/aunt-leah-b/blob/main/server/routers/__tests__/caseNote.test.ts**	**- Access control (e.g. a user can only see/edit case notes for clients/programs) they're allowed to access). - Validation of section content or entry dates.**
View Dashboard with relevant client data	**Unit Test**	**https://github.com/aunt-leahs-b-cms/aunt-leah-b/blob/main/server/routers/__tests__/dashboard.test.ts**	**1. Current dashboard search function is not working and needs to be debugged according to new change from client search page**

Acceptance Criteria

Client Search and Filtering

1. Search by name, ID, or contact information
   Status: 🟢 Met
   Evidence: Search by first_name and last_name implemented (clients.getAll, clients list page with search input)
2. Filter by program, case worker, enrollment date, status
   Status: 🟡 Partially Met
   Evidence: "mine" filter shows clients in user's programs (filter=mine URL param) here, active/inactive badge display here
   Missing: Specific program, case worker, date range filters | Plan: Add filter UI with dropdowns
3. Multiple filters can be applied simultaneously
   Status: 🟡Partially Met
   Evidence: Dashboard landing page provides such filter capabilities but not the browes all client
   Missing: Only search OR mine filter works, no UI for combining | Plan: Extend backend schema, build multi-filter component
4. Search results display in sortable table
   Status: 🟢 Met
   Evidence: Dashboard landing page shows a sortable client table
5. Save common filter combinations for quick access
   Status: 🔴 Not Met
   Missing: All functionality | Plan: Post-MVP - Add SavedFilter model, save/load UI

Client Overview Page

1. Client overview page displays goals, risks, and key demographics
   Status: 🟢 Met
   Evidence: Demographics displayed (name, DOB, age, gender, culture, marital status, contact, address, presenting issues), Goals button in header. Dashboard relegated to show the goal progress and risk status. here
2. Non-primary workers can view but not edit (read-only access)
   Status: 🟡 Partially Met
   Evidence: Permission system checks enrollment access via assertCanAccessClientById here, users can only view/edit clients in their assigned programs, clients list shows "Not One of Your Client" for restricted access here
   Missing: No primary worker concept, all users with access can edit | Plan: Add is_primary flag, implement read-only mode for non-primary workers

3. System clearly identifies primary case worker
   Status: 🔴 Not Met
   Missing: No primary worker field or display | Plan: Add primary_worker_id to Enrollment or Client model, display on client header

4. Recent activity/notes are summarized on overview
   Status: 🟢Met
   Evidence: Case notes sidebar shows sorted list with dates and authors, most recent note date in header, audit log shows last 10 changes with filtering for actual changes

5. Emergency contacts and critical information are prominently displayed
   Status: 🔴 Not Met
   Missing: No emergency contacts field, no critical info section | Plan: Add emergency contacts to schema, create prominent alert section for critical info.

Client Intake form

1. Clients can be enrolled into a program
   Status : 🟢 Met
   Evidence : https://aunt-leah-b.vercel.app/dashboard/intake
2. System doesn't create duplicate client records when re-enrolling a client into multiple programs
   Status : 🟢 Met
   Evidence : https://aunt-leah-b.vercel.app/dashboard/intake
3. Workers can only enroll clients into the programs they have access to
   Status : 🟢 Met
   Evidence : https://aunt-leah-b.vercel.app/dashboard/intake

4. Concurrency Tests - A race condition might occur if originally no client is there and two processes try to create the client first
   Status: 🔴 Not Met
   Missing: missing locks on db when its being mutated
   Plan: add locks to initiate transactions
5. Detailed Validation Tests - Validate date of birth and validate date of enrollment logic, test to see if we sanitize phone numbers
   Missing: validation for date fields
   Plan: add missing validation
   Status: 🔴 Not Met
6. Test to see if mismatching case or white space could result in client duplication
   Status: 🔴 Not Met
   Missing: Sanitanisation for input strings
   Plan: Normalise input strings by lowercase + remove whitespace

Goal Setting Page

1. Clients can set goals for each program they are in and set subtasks to achieve
   Status : 🟢 Met
   Evidence : https://aunt-leah-b.vercel.app/dashboard/goals/seed_client_naruto
2. Client can set a target completion date for each goal
   Status : 🟢 Met
   Evidence :https://aunt-leah-b.vercel.app/dashboard/goals/seed_client_naruto
3. Goal Tracking information is prominently displayed in the home page
   Status : 🟢 Met
   Evidence : https://aunt-leah-b.vercel.app/dashboard/intake
4. Client Progress can be measured by milestone completion
   Status : 🟢 Met
   Evidence : https://aunt-leah-b.vercel.app/dashboard

Authentication, Program Administration, and Security

1. Users can sign in through the auth flow and are forced through first-login onboarding when required
   Status: 🟢 Met
   Evidence: Auth flow is available at https://aunt-leah-b.vercel.app/auth/login. First-login onboarding is handled through https://aunt-leah-b.vercel.app/auth/first-login/password and https://aunt-leah-b.vercel.app/auth/first-login/2fa-setup.

2. MFA setup is required before non-super-admin users receive full access to protected procedures

Status: 🟢 Met
Evidence: MFA setup and verification are part of https://aunt-leah-b.vercel.app/auth/login, with challenge handling at https://aunt-leah-b.vercel.app/auth/2fa-challenge.

3. Admins can create new users with temporary passwords
   Status: 🟢 Met
   Evidence: Admin user creation is available at https://aunt-leah-b.vercel.app/dashboard/admin/users.
4. Admins can assign case workers to specific programs
   Status: 🟢 Met
   Evidence: Program assignment for workers is available at https://aunt-leah-b.vercel.app/dashboard/admin/users.
5. Admins can create new programs
   Status: 🟢 Met
   Evidence: Program creation is available at https://aunt-leah-b.vercel.app/dashboard/admin/programs.
6. Program-based security restricts workers to their assigned programs
   Status: 🟢 Met
   Evidence: Program-based access is enforced across protected dashboard routes such as https://aunt-leah-b.vercel.app/dashboard/clients, https://aunt-leah-b.vercel.app/dashboard/goals, https://aunt-leah-b.vercel.app/dashboard/case-notes, and https://aunt-leah-b.vercel.app/dashboard/intake

Case notes:

1. Create a case note for a client
   Status: 🟢 Met
   Evidence: https://aunt-leah-b.vercel.app/dashboard/case-notes/new
2. Edit Case note
   Status: 🟢 Met
   Evidence: https://aunt-leah-b.vercel.app/dashboard/case-notes/cmmmkap31000004if1wen153o
3. View Case Note
   Status: 🟢 Met
   Evidence: https://aunt-leah-b.vercel.app/dashboard/case-notes/cmmmkap31000004if1wen153o
4. Search Case Notes
   Status: 🟢 Met
   Evidence: https://aunt-leah-b.vercel.app/dashboard/case-notes

Dashboard (Statistics, Quick Actions, Client Table)

1. Display aggregated statistics (active clients, notes this week, goals on track, avg progress)
   Status: 🟢Met
   Evidence: https://aunt-leah-b.vercel.app/dashboard

2. Week-over-week note change comparison
   Status: 🟢 Met
   Evidence: https://aunt-leah-b.vercel.app/dashboard

3. Time-based greeting with user name (Good morning/afternoon/evening)
   Status: 🟢 Met
   Evidence: https://aunt-leah-b.vercel.app/dashboard

4. Quick action cards (New Client, Add Case Note, Export Data, Browse Clients)
   Status: 🟢 Met
   Evidence: https://aunt-leah-b.vercel.app/dashboard

5. My Cases table with client name, program, last note, goal progress, status
   Status: 🟢 Met
   Evidence: https://aunt-leah-b.vercel.app/dashboard

6. Sort cases by column (name, program, date, progress, status)
   Status: 🟢 Met
   Evidence: https://aunt-leah-b.vercel.app/dashboard

7. Filter cases by tab (All, On Track, Behind)
   Status: 🟢 Met
   Evidence: https://aunt-leah-b.vercel.app/dashboard

8. Toggle between table and grid view
   Status: 🟢 Met
   Evidence: https://aunt-leah-b.vercel.app/dashboard

9. Needs Attention sidebar with alerts (no note in 14+ days, behind on goals)
   Status: 🟢 Met
   Evidence: https://aunt-leah-b.vercel.app/dashboard

10. Attention items sorted by priority (behind first, then on-track with alerts)
    Status: 🟢 Met

Evidence: https://aunt-leah-b.vercel.app/dashboard

11. Role-based data scoping (workers see only their programs, admins see all)
    Status: 🟢 Met
    Evidence: https://aunt-leah-b.vercel.app/dashboard

12. Graceful fallbacks (empty DB, null names, zero goals, no programs)
    Status: 🟢 Met
    Evidence: **server/routers/tests/dashboard.test.ts**

MVP Testing

Testing Setup

- All unit tests run by cloning the repo and running pnpm test
- No external dependencies are required

Test Approach: Unit Testing with Vitest

- Mock-based unit tests for tRPC backend procedures
- Tests isolated router logic without real database calls
- Uses vi.mock to mock Prisma client operations
- Validates CRUD operations, user permissions, form validation, and audit logging etc
- Tests include both success and error scenarios

MVP Feature: Client Management (CRUD operations, search, permissions, audit logging)

Test Coverage:

Router Procedures Tested:

- list: Returns simple client list with basic fields (first_name, last_name, preferred_name)
- getAll: Filters clients by search query (first_name/last_name) and "mine" parameter, returns empty array when user has no program access
- getById: Returns client with enrollments, referrals, and audit log (limited to 10 entries), throws NOT_FOUND when client doesn't exist
- update: Updates client fields, creates audit log for changes, handles date comparisons correctly, skips audit log when no changes detected, updates presenting issues
- delete: Soft deletes client (sets is_active = false), creates audit log entry
- restore: Restores deleted client, enforces role-based permissions (admin/manager only), throws FORBIDDEN for unauthorized users

Validation Tests:

- First name/last name required (non-empty after trim)
- Date of birth constraints: must be after 1900-01-01 and before today
- Email format validation (regex pattern)
- Phone number validation (minimum 10 digits)

Permission Tests:

- "mine" filter respects user's program access
- Empty results when user has no program access
- Restore operation restricted to admin/manager roles

Audit Log Tests:

- Audit log created on update with actual changes
- No audit log when values unchanged
- Date comparison uses getTime() to prevent duplicate entries
- Audit log limited to 10 most recent entries

Evidence:
https://github.com/aunt-leahs-b-cms/aunt-leah-b/blob/main/server/routers/__tests__/client.test.ts

MVP Feature : Client Intake

Router functions tested

- Submit intake - returns a new enrollment record with client, program, referral info

Test Coverage

- Basic Scenario - No client detected, create client
- Prior client, reuse client record
- Basic Input validation - check valid emails
- Accept input when given only the required fields and none of the optional ones
- Reject if any required fields are missing

Evidence:
https://github.com/aunt-leahs-b-cms/aunt-leah-b/blob/main/server/routers/__tests__/intake.test.ts

MVP Feature : Authentication, MFA, Admin User Management, and Program Security

Router functions tested

- session
- changePasswordFirstLogin
- beginTwoFactorSetup
- verifyTwoFactorSetup
- verifyTwoFactorChallenge
- createUserWithTempPassword
- resetUserPasswordToTemp
- resetUserTwoFactor
- setWorkerProgramAccess
- doesUserHaveAccessToProgram
- getAccessibleProgramIds
- assertHasAccessToProgram

Test Coverage

- Returns null session values when user is not authenticated

- Returns onboarding state for authenticated users
- Verifies current password before first-login password change
- Updates password and returns MFA setup URI
- Handles MFA setup failure by returning a null URI
- Starts MFA setup successfully
- Verifies MFA setup and updates onboarding flags
- Verifies MFA challenge codes successfully
- Creates new users with temporary-password onboarding requirements
- Rejects duplicate email addresses during user creation
- Restricts admin-management actions to authorized roles
- Resets user passwords and clears previous 2FA state
- Resets user 2FA and forces setup again
- Updates worker program assignments through validated program IDs
- Rejects invalid program assignments
- Restricts program assignment to worker roles
- Returns allowed program IDs for workers
- Denies access when a worker does not have program permission

Evidence:
https://github.com/aunt-leahs-b-cms/aunt-leah-b/blob/main/server/routers/__tests__/auth.test.ts
https://github.com/aunt-leahs-b-cms/aunt-leah-b/blob/main/server/routers/__tests__/adminAuth.test.ts
https://github.com/aunt-leah-b-cms/aunt-leah-b/blob/main/server/helper/__tests__/programAccess.test.ts

MVP Feature: Goal Setting

Router functions tested

Retrieve all goal info for all client for all enrollments (list)
- Returns all active goals

Retrieve all goal info for a client for all enrollments (getByClientId)
- Throw if invalid clientId
- Returns [] if no enrollments exist for a client

createGoal
- Creates goal is no active goal is present for an enrollment
- Throws NOT_FOUND if enrollment is missing
- Throws CONFLICT when a goal is present already

updateGoal
- Updates a goals name
- Sets completion date
- Clears completion date when null

- Rejects empty goal name

addSteps

- Adds step at idx 0 when no steps exist
- Appends step to last index otherwise

updateStep

- Updates step name
- Toggle incomplete status
- Update incompletion without changing name and vice-versa
- Update both together

Reorder Steps

- updates idx for each provided step
- Handles empty step arrays

deleteSteps - set is_active to false

Evidence:
https://github.com/aunt-leahs-b-cms/aunt-leah-b/blob/main/server/routers/__tests__/goal..test.ts

MVP Feature: View/Edit/Create Case Notes

Test Coverage:

Router procedures tested

- list:

- Returns case notes with default filters and applies optional filters when provided
- Response shape includes client (id, name, preferred_name), program, and sections.

getById

- Returns case note with enrollment, client, program, author, and sections when found.
- Throws NOT_FOUND when the case note does not exist or is inactive.

- create:

- Creates a draft case note when an active enrollment exists for the given client/program.
- Resolves enrollment by client_id, program_id, status active, latest enrollment_date.
- Throws NOT_FOUND when no active enrollment exists; does not call create.

- update:

- Updates section fields.
- Throws NOT_FOUND when case note does not exist or is inactive.

- publish:

- Sets status to published for a draft case note.
- Throws NOT_FOUND when case note does not exist.
- Throws BAD_REQUEST when case note is already published.

- delete:

- Soft-deletes by setting is_active = false.
- Throws NOT_FOUND when case note does not exist.

MVP Feature: Dashboard (Statistics, Quick Actions, Client Table)

Test Coverage:

Router procedures tested:

- getData:
  - Returns correct shape with userName, currentProgram, stats, cases, attentionItems
  - Resolves user first name from database, falls back to "User" if not found
  - Falls back to "All Programs" when no programs exist
  - Counts active clients via enrollment grouping
  - Counts published notes for current week (last 7 days) and last week (days 7–14)
  - Calculates week-over-week note change (including negative values)
  - Calculates goals on track vs total goals, completed goals count as 100%
  - Computes average goal progress, returns 0% when zero goals exist
  - Builds case rows with correct initials, full name, program, goal progress, and derived status
  - Status derivation: on_track when progress >= 25%, behind when < 25%
  - Generates alerts: "No note in X days" when last note >= 14 days old, "No notes recorded" when none exist
  - Merges goals across multiple enrollments for the same client

Permission Tests:

- Non-admin workers only see data from programs they have access to via WorkerProgramAccess
- Admin users see all programs

Attention Items Tests:

- Includes clients that are behind OR have stale note alerts
- Sorted by priority: behind clients first, then on-track with alerts

Edge Case Tests:

- Empty database returns all zeros and empty arrays gracefully
- Null client names default to "Unknown" / "??"
- Database errors propagate correctly

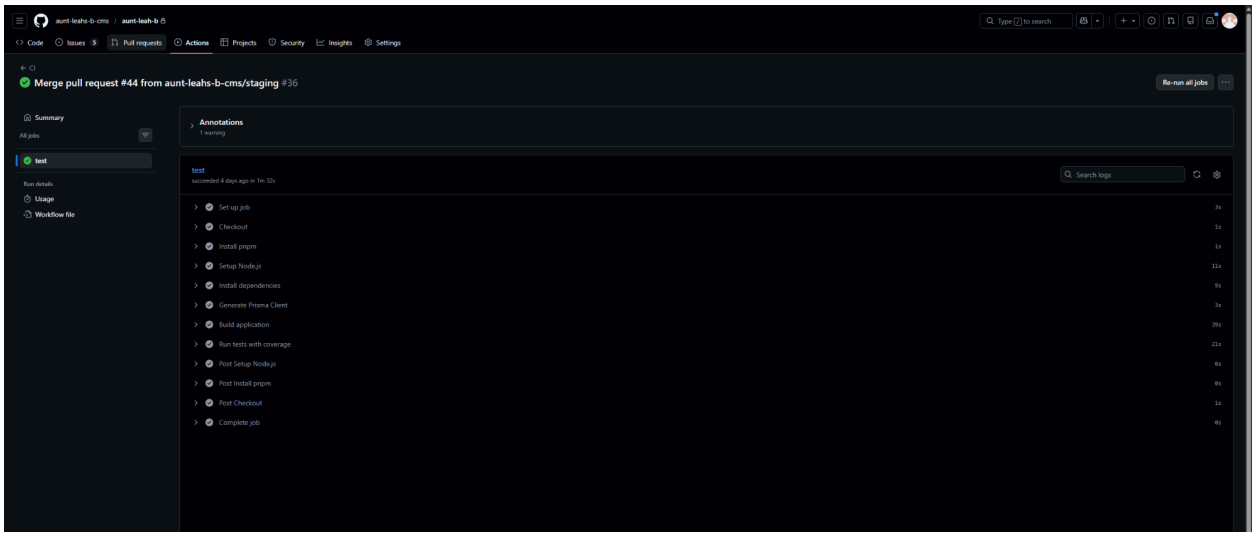
Evidence:
https://github.com/aunt-leahs-b-cms/aunt-leah-b/blob/main/server/routers/__tests__/dashboard.test.ts

CI/CD

Owner: David Falade (Dev-ops Lead)

We currently have automated unit and router tests as well as a minimum test coverage percentage of 70%. We plan on adding automated integration tests by march 30th.
All tests as well as the test coverage checks must pass before any merges to main or staging unless in exceptional circumstances on behest of the Dev-op lead.

Build Page:
https://github.com/aunt-leahs-b-cms/aunt-leah-b/actions/runs/22980495599/job/66719004035



Accessibility Testing

Owner - Shreyan Das
Pages/ Flows Checked:

- Editing Client Details
- Adding a client

Issues found

- The background and text colour on the client/case notes page doesn't have enough contrast, making it harder for colourblind people to read
- The responsive cues on the client page are too subtle, making it harder to see when its being triggered
- The text on the search navbar clashes with the black background, decreasing readability
- Optional markers on the intake form is too faint, compared to the light background, decreasing readability

Tickets made

name	owner	status
Improve contrast for background and text on client page	Michael Sam	to do
Make responsive cues more visible on client page	Michael Sam	to do
Increase contrast for search navbar text	Kyaw Min Oo	to do
Make optional markers on intake form more visible	Shreyan Das	to do
Improve contrast for background and text on case notes page	David Falade	to do

Evidence

- Manual review done by visually impaired team member

Peer Testing Results and Follow-up

Context

Date: February 23rd, 2026
Number of Testers: 7
Version/Build Tested: Staging branch with locally running build and locally running database.

Feedback Form Questions

Login Page:
Tester Tries:
-normal successful admin login
-normal successful user login
-unsuccessful login (incorrect password/username)
-without being logged in tries to access /clients page (should reject since not logged in)

Dashboard Page:
Tester Tries:
-interacting with the dashboard and we observe time to understand UI
-interacting with all the links to connect to the various pages

Intake Page:
Tester Tries:
-normal successful intake
-unsuccessful intake due to missing required fields

Client Page (list view):
Tester Tries:
-tester does a failed search (client not found)
-tester searches for the client they added in the intake
-tester clicks the newly added client from the clients page

Specific Client Page
Tester Tries:
-normal edit and save to the client they added
-verify that the change was noted in the audit log
-click edit, tester didn't edit anything and then they save (check if no audit log appeared)

Case Notes Page
Tester Tries:
-adding a case note and saves it as a draft
-verifying that it was added to the right client
-publishing the case note

Short Summary of Results
The peer testing session on February 23, 2026 identified 9 total issues across the application. The most critical problems centered around core functionality: client creation was completely broken (#001), editing client records was impossible (#002), and case note navigation was non-functional (#003). These issues would prevent users from performing essential daily tasks. Additional major findings included a dead links in the case notes history section (#004) and a missing dashboard tab (#005). Four minor issues were also identified related to search behavior, user feedback, form labeling, and timestamp display. All issues have been fixed via pull requests.

Issues/Ticket Links
Note: The term, User Feedback, as used in the following encompasses, all bugs/issues reported by our peer testers for a given page of our application.

- User Feedback — Auth Pages
- Search Clients + Filtering
- User Feedback — Client Page
- User Feedback — Intake Page
- User Feedback — Case Notes

What we will do differently for the final peer testing session
For our next peer testing session, we will focus on better preparedness through more rigorous pre-testing, specifically prioritizing End-to-End testing beforehand to catch unique edge cases

early. As a team, the majority of us have limited experience with UI/UX, but we recognize that hands-on exposure is the best way to build that muscle. To push ourselves, we are proposing a mini challenge: end the final peer testing session with only cosmetic issues remaining. This means no critical or major bugs, just minor visual or usability polish. We believe this goal will force us to think more like end users by anticipating workflow friction and ultimately deliver a much more polished product.